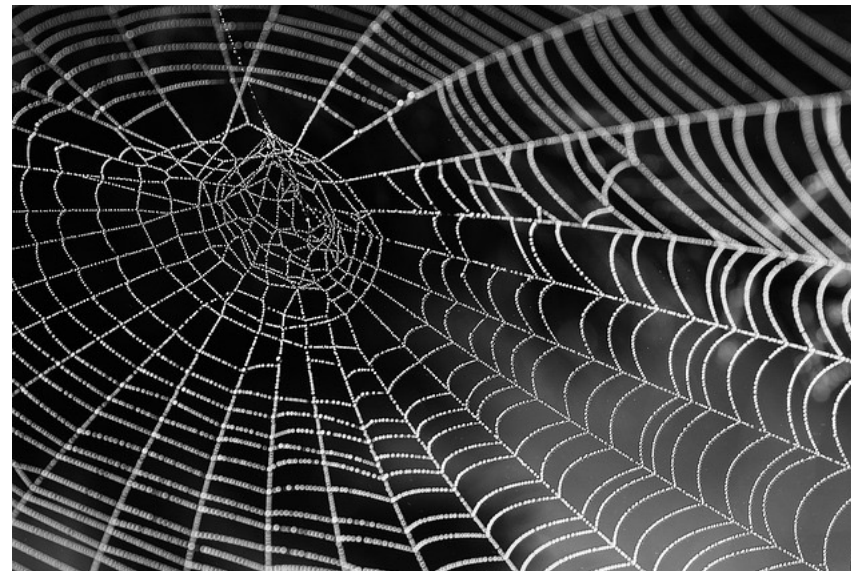


Prijediplomski studij

Računarstvo

Ak. god. 2025./2026.

Komunikacijske mreže 2



7. Web-usluge temeljene na prijenosu prikaza stanja resursa (REST)

Representational State Transfer (REST)

Web-usluge temeljene prijenosu prikaza stanja resursa (REST)

- REST (*Representational State Transfer*)
- pojmovi: *The REST Way* ili *RESTful services*
- pojam je skovao Roy Fielding u svojoj doktorskoj disertaciji
- nije standard već arhitekturni stil
- sve se temelji na resursima koji su predstavljani URL-ovima:
 - `http://localhost:8080/users` - popis svih korisnika
 - `http://localhost:8080/users/1` - korisnik s identifikatorom 1
- koristi protokol: HTTP (GET, POST, PUT, DELETE, PATCH)
- koristi podatke: JSON, XML, "sirovi" (npr. za slike, video)
- bez stanja (*stateless*), priručni spremnik (*cache*)

Svojstva metoda protokola HTTP

- Svojstva:
 - Sigurna (*safe*) - bez posljedica za podatke
 - *Idempotentna* - može se izvršavati više puta s istim rezultatom
 - Spremljiva (*cacheable*) - može se privremeno spremiti odgovor
- Metode:
 - GET - sigurna, idempotentna, spremljiva
 - HEAD - sigurna, idempotentna, spremljiva
 - OPTIONS - sigurna, idempotentna
 - POST - nema niti jedno svojstvo, ali se može napraviti idempotentna
 - PUT - idempotentna
 - DELETE - idempotentna
 - PATCH - nije uvijek idempotentna (primjer brojača)

Tipično korišćenje usluga REST

- za *Create, Read, Update and Delete* (CRUD)

HTTP	CRUD
POST	Create, (Overwrite/Replace)
GET	Read
PUT	Update, (Create, Delete)
DELETE	Delete
PATCH	Partial update

CQS (Command-query separation)

- umjesto CRUD-a bolje je koristiti CQS
- obrazac koji je prvi uveo Bertrand Meyer
- svaka metoda može biti ili:
 - komandna – koja zapisuje nešto ili mijenja stanje (*side effect*), ne vraća rezultat ili
 - upit – koja samo čita i vraća podatke, ali ne mijenja stanje
- nije uvijek moguće to napraviti, npr. uzimanje podatka sa stoga

Tablica napravljene usluge (račun mob. tk. usluge)

resurs	metoda	što se šalje	svrha
/users	POST	korisnik	stvara novog korisnika
	GET		vraća popis korisnika
/user/{id}	GET		vraća korisnika s ID-om
	DELETE		briše korisnika
	PUT	korisnik	mijenja podatke o korisniku
/user/{id}/{year}/{month}	GET		vraća račun korisnika za godinu i mjesec
/user/{id}/basicTariff	PUT	podaci o tarifi	postavlja osnovnu tarifu
/user/{id}/additionalTariff/{year}/{month}	POST	dodatna tarifa	stvara dodanu tarifu za mjesec
/user/{id}/timeTariff	POST	vremenska tarifa	stvara vremensku tarifu
/user/{id}/tariff/{year}/{month}	GET		dohaća popis tarifa za korisnika i za mjesec
/user/{id}/canStartCall	GET		provjera mogućnosti pozivanja
/user/{id}/canSendMessage	GET		provjera mogućnosti slanja poruke
/user/{id}/canStartDataSession	GET		provjera mogućnosti pokretanja podatkovne sjednice
/consumingRecord	POST	zapis potrošnje	dodaje novi zapis potrošnje

OpenAPI

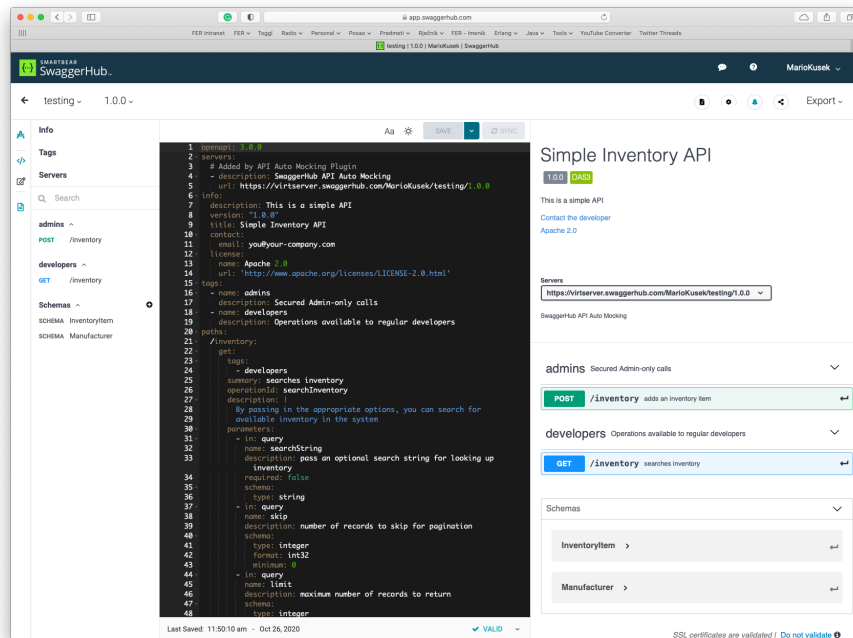


- [OpenAPI inicijativa](#) (konzorcij)
 - osnovana 2016. u sklopu Linux Foundation
 - OpenAPI specification (bivši Swagger 2.0 spec. → OpenAPI – OASx)
 - aktualna verzija [3.2.0](#) ([specifikacija na GitHubu](#))
- mogućnosti:
 - neovisno o tehnologijama koje se koriste (programski jezici ili radni okviri)
 - dokument definiran u JSON-u ili YAML-u
 - [OpenAPI Tools](#)
 - konverteri, validatori, GUI *editori*, lažne usluge (poslužitelji), generatori dokumentacije
 - [OpenAPI CodeGen](#)
 - generiraju kostur programa:
 - klijent – 20+ jezika i knjižnica
 - poslužitelj – 40+ jezika i radnih okvira

Uređivači (*editor*)

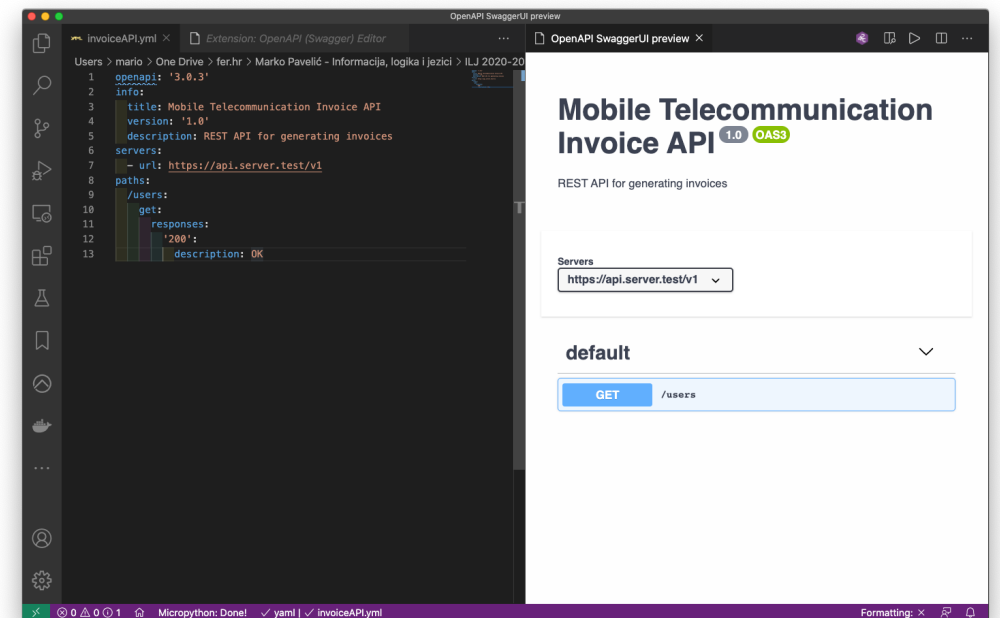
Online – SwaggerHub

<https://swagger.io/tools/swaggerhub/>



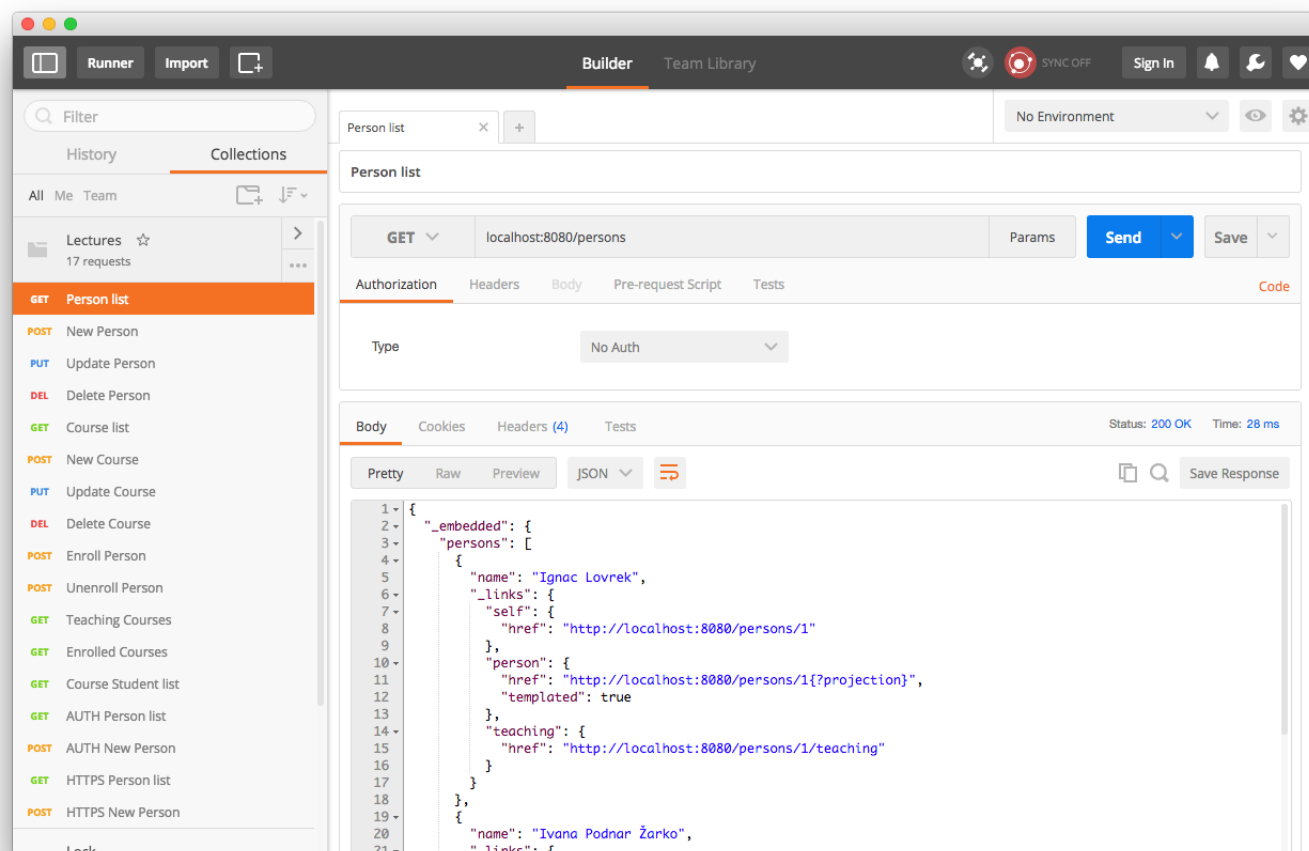
VS Code

OpenAPI (Swagger) Editor



Primjer dohvaćanja

- Chrome extensions (<https://chrome.google.com/webstore/search/rest>)
- Aplikacija [Postman](#)
 - Alternativa otvorenog koda: [Bruno](#)
- Alati u terminalu:
 - [curl](#)
 - [http](#)



Primjer upita i odgovora - stvaranje resursa (1)

post:

description: Creates new user

requestBody:

required: true

content:

application/json:

schema:

\$ref: "#/components/schemas/user"

responses:

'201':

description: User created

headers:

Location:

description: location of the created user

schema:

type: string

format: uri

example: http://localhost/users/{userId}

POST

/users

Creates new user

Parameters

Try it out

No parameters

Request body required

application/json

Example Value

Schema

```
{
  "firstName": "Pero",
  "lastName": "Perić",
  "address": "Unska 3, 10000 Zagreb"
}
```

Responses

Code	Description	Links
201	User created	No links

Headers:

Name	Description	Type
Location	location of the created user	string Example: http://localhost/users/{userId}

Primjer upita i odgovora - stvaranje resursa (2)

user:

type: object

properties:

firstName:

type: string

example: Pero

lastName:

type: string

example: Perić

address:

type: string

example: Unska 3, 10000 Zagreb

Schemas

```
user {  
  firstName      string  
                  example: Pero  
  lastName       string  
                  example: Perić  
  address        string  
                  example: Unska 3, 10000 Zagreb  
}
```

Primjer upita i odgovora - stvaranje resursa (3)

- Tijelo upita:

```
{  
  "firstName": "Pero",  
  "lastName": "Perić",  
  "address": "Unska 3, 10000 Zagreb"  
}
```

```
curl -X POST "http://localhost:8080/users" \  
-H "accept: */*" -H "Content-Type: application/json" \  
-d '{  
  "firstName": "Pero",  
  "lastName": "Perić",  
  "address": "Unska 3, 10000 Zagreb"  
}'
```

Primjer upita i odgovora – dohvaćanje popisa (1)

```
/users:  
  get:  
    responses:  
      '200':  
        description: List of users  
        content:  
          application/json:  
            schema:  
              type: array  
              items:  
                $ref: "#/components/schemas/shortUser"  
        description: Gets list of users
```

The image shows a screenshot of an API Explorer interface for the `GET /users` endpoint. The interface is divided into several sections:

- Method and Path:** A blue button labeled `GET` and the path `/users`.
- Description:** A text box containing "Gets list of users".
- Parameters:** A section with a "Try it out" button and the text "No parameters".
- Responses:** A table with columns "Code", "Description", and "Links". It shows a response with code "200", description "List of users", and "No links".
- Media Type:** A dropdown menu labeled "Media type" with "application/json" selected. Below it, a note says "Controls Accept header." and links for "Example Value" and "Schema" are provided.
- Example Value:** A dark box displaying a JSON array:

```
[ { "id": 42, "name": "Pero Perić" } ]
```

Primjer upita i odgovora – dohvaćanje popisa (2)

components:

schemas:

shortUser:

type: object

properties:

id:

type: integer

example: 42

name:

type: string

example: Pero Perić

Schemas 

```
shortUser ▾ {  
  id                integer  
                   example: 42  
  name              string  
                   example: Pero Perić  
}
```

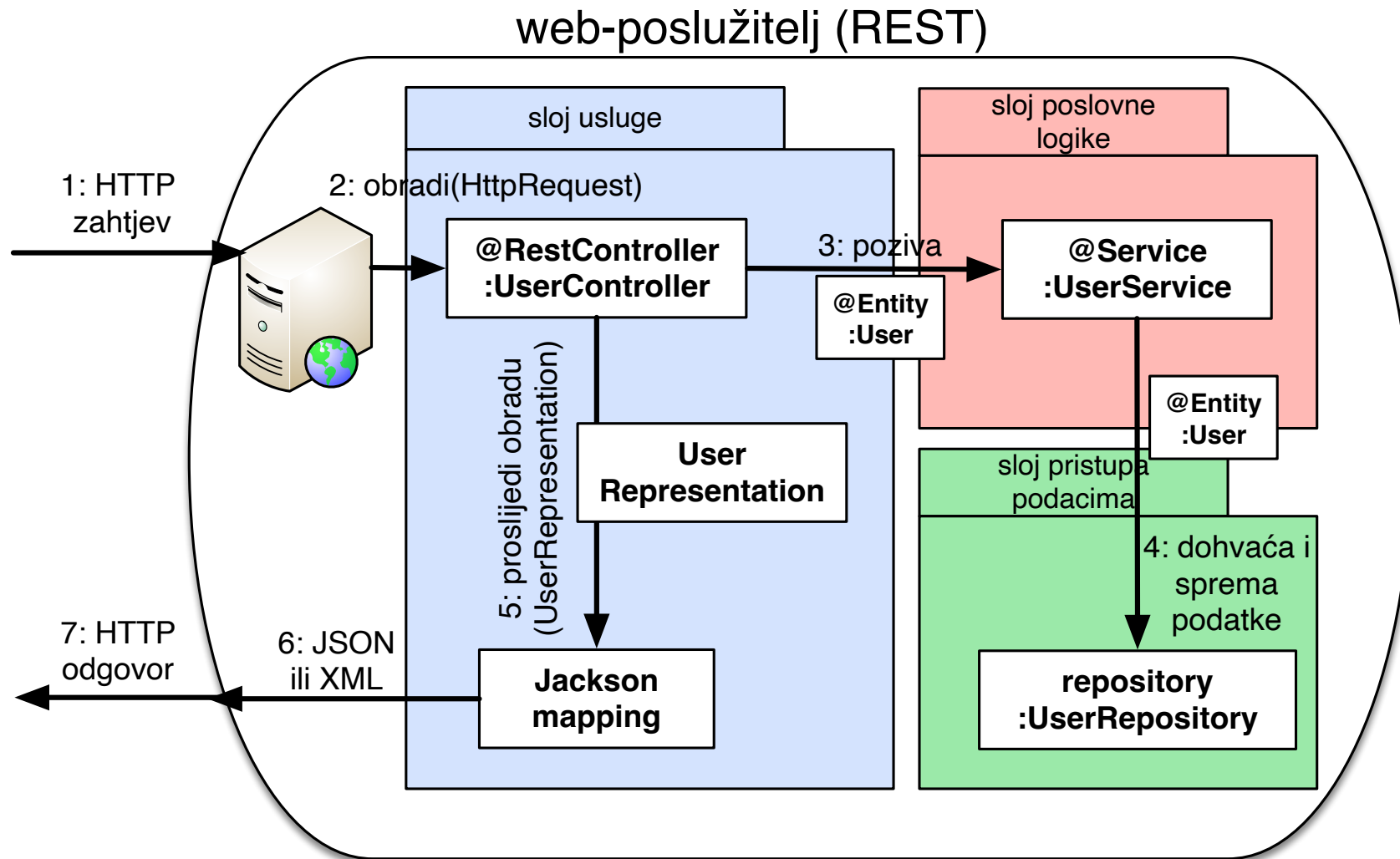
API

GET	/users
POST	/users
GET	/user/{id}
DELETE	/user/{id}
PUT	/user/{id}
GET	/user/{id}/{year}/{month}
PUT	/user/{id}/basicTariff
POST	/user/{id}/additionalTariff/{year}/{month}
POST	/user/{id}/timeTariff
GET	/user/{id}/tariff/{year}/{month}
GET	/user/{id}/canStartCall
GET	/user/{id}/canSendMessage
GET	/user/{id}/canSStartDataSession
POST	/consumingRecord

Schemas
user >
shortUser >
tariffElements >
tariff >
additionalTariff >
timeTariff >
consumedTariff >
invoice >
consumingRecord >



Arhitektura REST web-usluge u Springu



Stvaranje Spring projekta

- otvoriti stranicu <https://start.spring.io>
- Odabrati tip: Gradle - Groovy
- popuniti:
 - Name: invoice
 - Packaging: Jar
 - Java Version: 21
 - Language: Java
 - Group: hr.fer.km2
 - Artifact: invoice
 - Package: hr.fer.km2.invoice
- odabrati: Spring Web, Spring Boot Actuator, Spring Boot DevTools
- klik na Generate Project
- Eclipse → import Gradle project

REST kontroleri

- Klasa se označi sa `@RestController`
- Oznake metoda u Javi:
 - `@GetMapping("uri")` – HTTP GET
 - `@PostMapping`, `@PutMapping`, `@DeleteMapping`, `@PatchMapping`
- Parametri metode:
 - Parametri uri-ja:
 - Npr. `"/user/{id}"`
 - oznaka argumenta metode: `@PathVariable("id")`
 - `@RequestBody` – sadržaj zahtjeva kod metoda POST ili PUT
- Iznimke se mogu označiti:
 - `@ResponseStatus(value = HttpStatus.NOT_FOUND)`
- Rezultat metode
 - Objekt koji se pretvara u JSON ili
 - `ResponseEntity<O>` - parametriziran prema objektu koji se vraća
 - Ima metode za podešavanje statusa npr. za 204 ili 201 i za podešavanje zaglavlja

Klase ShortUser, User

```
public record ShortUser(long id, String name) {  
}
```

```
public record User(  
    Long id,  
    String firstName,  
    String lastName,  
    String address)  
{  
    public static User empty() {  
        return new User(null, null, null, null);  
    }  
}
```

Implementacija kontrolera

```
@RestController
public class UserController {

    private UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    @GetMapping("/users")
    public Collection<ShortUser> allUsers() {
        return userService.getAll().stream().map(u -> toShortUser(u))
            .collect(Collectors.toList());
    }

    private ShortUser toShortUser(User user) {
        return new ShortUser(user.getId(),
            String.format("%s %s", user.firstName(), user.lastName()));
    }
}
```

Konfiguracija UserServicea

@Configuration

```
public class DomainConfiguration {
```

```
    @Bean
```

```
    public UserService userService(UserRepository repo) {  
        return new UserService(repo);  
    }
```

```
    @Bean
```

```
    public UserRepository userRepository() {  
        return new InMemoryUserRepository();  
    }
```

```
}
```

Klasa UserRepresentation

```
public record UserRepresentation (  
    String firstName,  
    String lastName,  
    String address) {  
}
```

Stvaranje korisnika

```
@RestController()  
public class UserController {  
    ...  
    @PostMapping("/users")  
    public ResponseEntity<Void> addUser(@RequestBody UserRepresentation userRepresentation)  
        throws URISyntaxException {  
        long userId = userService.addUser(toUser(userRepresentation));  
        return ResponseEntity.created(new URI("/user/" + userId)).build();  
    }  
  
    private User toUser(UserRepresentation userRepresentation) {  
        return toUser(null, userRepresentation);  
    }  
  
    private User toUser(Long userId, UserRepresentation userRepresentation) {  
        return new User(userId, userRepresentation.firstName(), userRepresentation.lastName(),  
            userRepresentation.address());  
    }  
}
```

Dohvaćanje korisnika

```
@RestController()  
public class UserController {  
    ...  
    @GetMapping("/user/{id}")  
    public ResponseEntity<UserRepresentation> getUser(@PathVariable("id") Long userId) {  
        return userService.getUser(userId).map(this::toUserRepresentation)  
            .map(ur -> ResponseEntity.ok(ur))  
            .orElseGet(() -> ResponseEntity.notFound().build());  
    }  
  
    private UserRepresentation toUserRepresentation(User user) {  
        return new UserRepresentation(user.firstName(), user.lastName(), user.address());  
    }  
}
```

Generiranje klijenta iz OpenAPI-a (1)

- Github projekt: <https://github.com/OpenAPITools/openapi-generator>
- skinuti `swagger-codegen-cli.jar` (provjeriti zadnju verziju [ovdje](#))
- za pregledavanje popisa generatora
`java -jar openapi-generator-cli.jar list`
- za pregledavanje opcija generatora
`java -jar openapi-generator-cli.jar help generate`
- za pregledavanje dodatnih opcija generatora java pokrenuti:
`java -jar openapi-generator-cli.jar config-help -g java`

Generiranje klijenta iz OpenAPI-a (2)

- Bitne opcije:
 - `-g` ime generatora
 - `--library` – odabir klijentskih biblioteka (`native`)
 - `-i` ulazna datoteka (URL ili *path*)
 - imena paketa generiranih klasa `--api-package`, `--model-package`, `--invoker-package`
 - svojstva projekta `--group-id`, `--artifact-id`, `--artifact-version`
 - izlazni direktorij `-o`

Generiranje klijenta iz OpenAPI-a (3)

- Primjer pokretanja:

```
java -jar openapi-generator-cli.jar generate \  
  -i invoice/invoiceAPI.yml \  
  --api-package hr.fer.km2.invoice.client.api \  
  --model-package hr.fer.km2.invoice.client.model \  
  --invoker-package hr.fer.km2.invoice.client.invoker \  
  --group-id hr.fer.km2 \  
  --artifact-id invoice-api-client \  
  --artifact-version 0.0.1-SNAPSHOT \  
  -g java \  
  --library native \  
  -o invoice-api-client
```

Podešavanje dobre verzije gradlea

- U projektu pokrenuti ažuriranje verzije gradlea:
`gradle wrapper --gradle-version 8.12.1`
- U `build.gradle` popraviti verziju jave na 21

Klasa:

DefaultApi

All URIs are relative to <http://localhost:8080>

Method	HTTP request	Description
<u>consumingRecordPost</u>	POST /consumingRecord	
<u>consumingRecordPostWithHttpInfo</u>	POST /consumingRecord	
<u>userIdBasicTariffPut</u>	PUT /user/{id}/basicTariff	
<u>userIdBasicTariffPutWithHttpInfo</u>	PUT /user/{id}/basicTariff	
<u>userIdCanSendMessageGet</u>	GET /user/{id}/canSendMessage	
<u>userIdCanSendMessageGetWithHttpInfo</u>	GET /user/{id}/canSendMessage	
<u>userIdCanStartCallGet</u>	GET /user/{id}/canStartCall	
<u>userIdCanStartCallGetWithHttpInfo</u>	GET /user/{id}/canStartCall	
<u>userIdCanStartDataSessionGet</u>	GET /user/{id}/canStartDataSession	
<u>userIdCanStartDataSessionGetWithHttpInfo</u>	GET /user/{id}/canStartDataSession	
<u>userIdDelete</u>	DELETE /user/{id}	
<u>userIdDeleteWithHttpInfo</u>	DELETE /user/{id}	
<u>userIdGet</u>	GET /user/{id}	
<u>userIdGetWithHttpInfo</u>	GET /user/{id}	
<u>userIdMonthlyTariffPost</u>	POST /user/{id}/monthlyTariff	
<u>userIdMonthlyTariffPostWithHttpInfo</u>	POST /user/{id}/monthlyTariff	



Primjer korištenja

```
ApiClient defaultClient = Configuration.getDefaultApiClient();  
defaultClient.updateBaseUri("http://localhost:8080");
```

```
DefaultApi apiInstance = new DefaultApi(defaultClient);
```

```
try {  
    List<ShortUser> users = apiInstance.usersGet();  
    for (ShortUser user : users) {  
        System.out.println(user);  
    }  
} catch (ApiException e) {  
    e.printStackTrace();  
}
```

Postavljanje web-aplikacije na poslužitelj

- Postupak:
 - Spajanje na poslužitelj
 - Prijenos datoteka
 - Konfiguriranje za produkciju
 - Prava pokretanja i slušanja na vratima 80
 - Konfiguriranje i spremanje logova
 - Automatsko pokretanje kod *restarta* poslužitelja

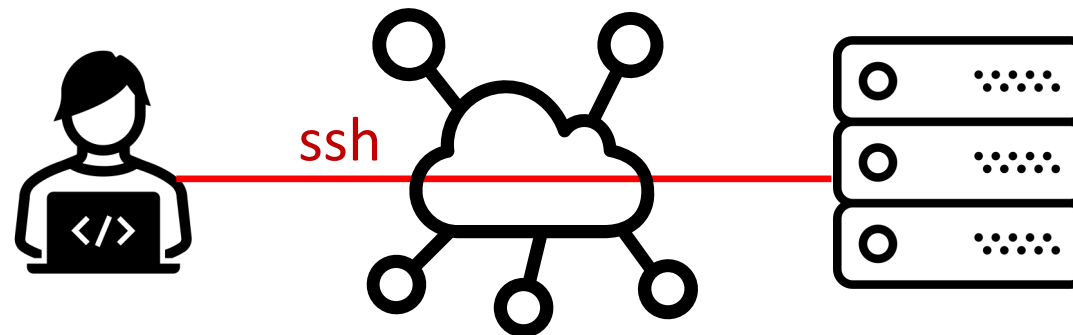
Spajanje na poslužitelj – SSH

- SSH je servis za udaljeno spajanje na računalo
- Koristi sigurne mehanizme spajanja (zaštićena veza)
- OpenSSH – najpoznatija implementacija, otvorenog koda
- Preko njega je moguće:
 - Izvršavanje komandi na udaljenom računalu, prenositi datoteke, tuneliranje
- Načini autentifikacije:
 - Korisničko ime i lozinka – nije toliko sigurno
 - Ključevi - poželjno
- Alati:
 - Unix/Linux/Windows 10+ – komanda ssh i scp
 - Alternativa za Windowse: aplikacija [putty](#), [winscp](#)

Načini korištenja

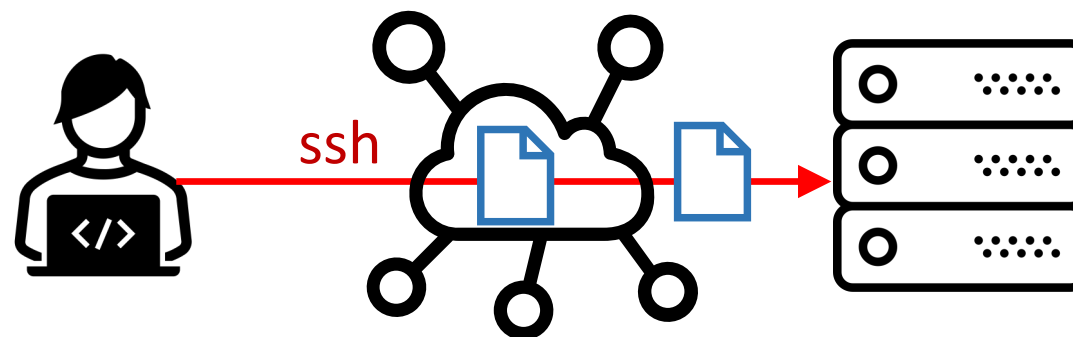
- udaljeni pristup:

```
ssh user@host
```



- kopiranje na udaljeno računalo:

```
scp src_path user@host:dest_path
```



Pokretanje aplikacije da sluša na vratima 80

- Potrebno je postaviti konfiguraciju u `application.properties`:
`server.port=80`

- Pokretanje:

```
sudo java -jar invoice.jar  
--spring.config.location=file:/home/mario/invoice/application.properties
```

- Javlja se pogreška:

```
...  
*****  
APPLICATION FAILED TO START  
*****
```

Description:

Web server failed to start. Port 80 was already in use.

Detekcija što sluša na vratima

- Komande:
 - ss, netstat ili lsof – za detekciju koji proces sluša na kojim vratima
 - ps – za pregled kako je pokrenut proces
 - sudo – za pokretanje komande kao root
 - less – za interaktivni ispis
 - grep – za filtriranje
- Sistemski procesi:
 - Za pokretanje poslužitelja (servisa) se koristi *systemd - System and Service Manager*
 - Za pregled što je pokrenuto koristimo komandu `systemd-cgls`
 - Komanda `systemctl` za upravljanje sa servisima (*unit*):
 - `systemctl <komanda> <unit>`
 - Komande: `status`, `stop`, `start`, `restart`, `reload`, `enable`, `disable`
 - Komanda za detekciju problema (ispisa) koji vidi *systemd*:
 - `journalctl -u invoice.service`

systemd

- Konfiguracijska datoteka: `invoice.service` se nalazi `/etc/systemd/system/invoice.service`

User=java

Group=java

WorkingDirectory=/opt/invoice

EnvironmentFile=-/etc/default/invoice

ExecStart=/usr/bin/authbind --deep /usr/bin/java -jar /opt/invoice/invoice.jar
--spring.config.location=file:/etc/invoice/application.properties

Restart=on-failure

RestartSec=5

[Install]

WantedBy=multi-user.target

logiranje

- SpringBoot koristi [logback](#) library za logiranje
- Zahtjevi:
 - Posebna datoteka za našu aplikaciju
 - Ne želimo zapuniti disk logovima
 - Potrebno je ograničiti količinu logova
 - Strategija:
 - automatsko mijenjanje (rotiranje) aktivne log-datoteke
 - Kada se napuni (maksimalna veličina 10MB) onda se prelazi na drugu datoteku
 - Imamo određeni broj datoteka (npr. 10)
- Primjer konfiguracije je u datoteci `logback-spring.xml`

Konfiguracija logiranja u SpringBootu

- Popravimo `application.properties`:

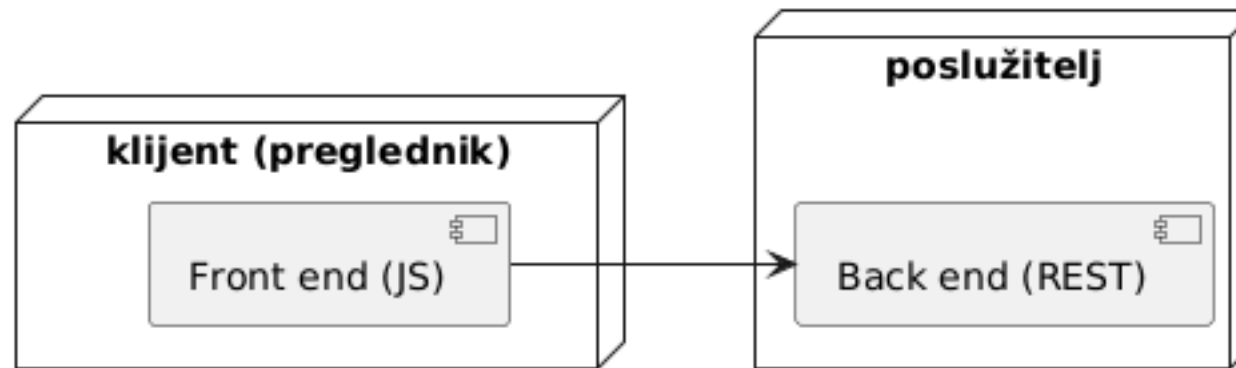
```
server.port=80
```

```
# external Logback config
```

```
logging.config=file:/etc/invoice/logback-spring.xml
```

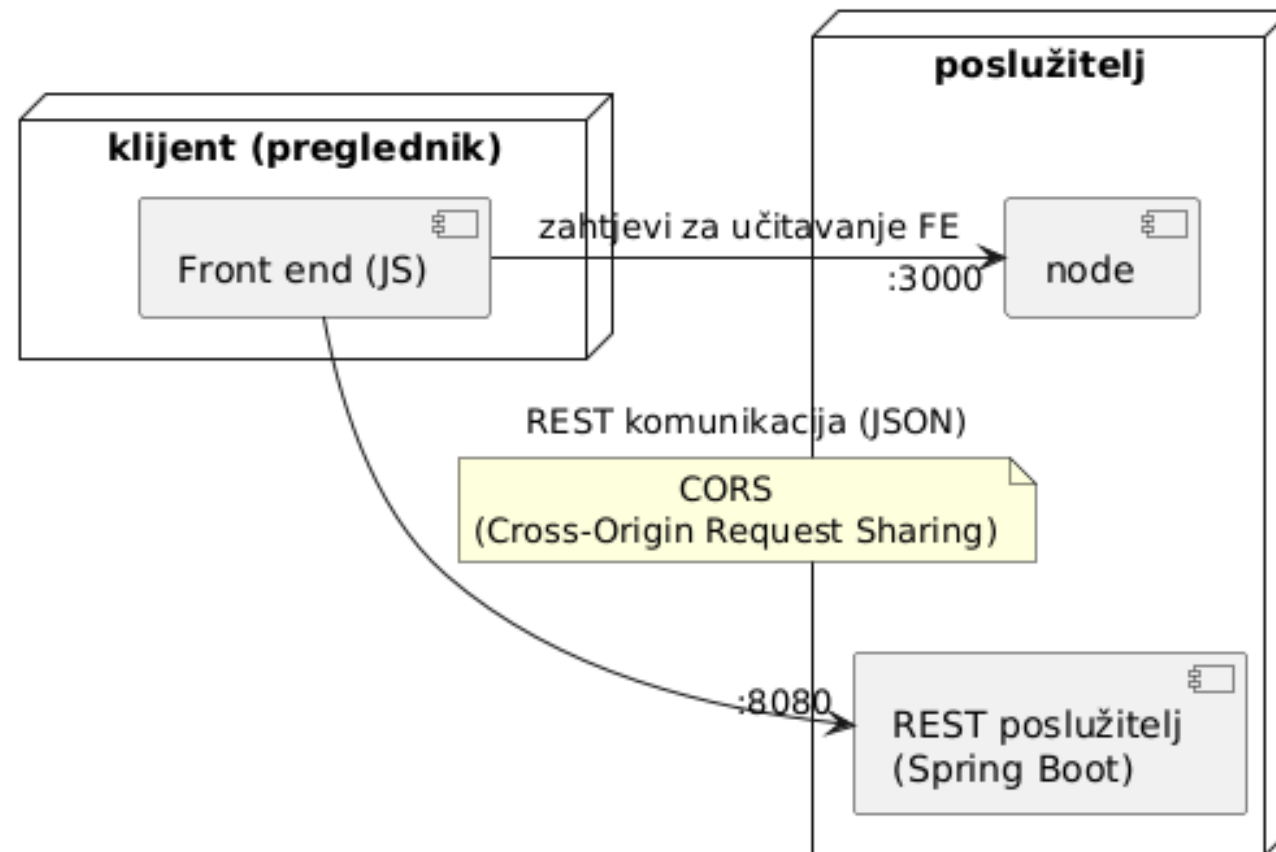
Web-aplikacije (moderne)

- Sastoje se od:
 - *front end* – dio koji se izvršava u pregledniku
 - *backend* – dio koji se izvršava na poslužitelju
- U ovom slučaju poslužitelj poslužuje FE JS i ima REST na BE



Web-aplikacije – razvojna (dev) konfiguracija

- Potrebno je:
 - automatsko ponovno učitavanje prilikom promjene koda na FE



Web-aplikacije – produkcijska (prod) konfiguracija

- Sve (FE i BE) se poslužuje s jednog poslužitelja

